

UNIVERSITÉ SORBONNE PARIS NORD
IUT DE VILLETANEUSE
Département Informatique

R2.06 – Exploitation d'une base de données

RAPPORT DE SYNTHÈSE :

Groupe Mars

Anass FATHI

Destinataire : M.BRINGUIER

Sous la direction de : M.BRINGUIER

Date de rendu : 08/06/2026

Table des Matières :

I – Introduction	3 page
II – Préparation de l’environnement	3 page
II – Requêtes SQL d’exploitation	4 page
IV – Création de vues SQL	7 page
V – Construction du dashboard Grafana	8 page
VI – Sauvegarde PostgreSQL	9 page
VII – Conclusion	10 page

I – Introduction

Dans un monde centré sur les données, les bases de données SQL exécutent l'affichage. Les éditeurs de logiciels, les banques, les agences gouvernementales, les universités et les organisations de différents secteurs d'activité s'appuient sur des bases de données relationnelles pour stocker des enregistrements, gérer les données et effectuer des opérations commerciales importantes. C'est pourquoi mon dossier portfolio introduit mes capacités à mettre en œuvre cette chaîne de traitement de son architecture vers l'exploitation de ses données stockées .

Pour cela , nous devons mobiliser quelques outils afin de mener à bien notre projet :

- Linux , Héberge les services PostgreSQL et Grafana
- PostgreSQL ,Stocke, organise et exploite les données
- SQL , Interroge, transforme et agrège les données
- Grafana , Visualise les résultats et produit les dashboards

II – Préparation de l'environnement

Pour début notre projet de la manière la plus efficace il nous faut créer l'environnement adéquats qui accueillera notre base de données .Une base de données correctement conçue nous permettra d'accéder à des informations à jour et précises. Étant donné qu'une conception correcte est essentielle pour atteindre nos objectifs dans l'utilisation d'une base de données, il est judicieux de nous préparer pour celle- ci . En fin de compte, nous serons beaucoup plus susceptibles de nous retrouver avec une base de données qui répond à nos besoins et peut facilement s'adapter au changement .

Notre première étape est de mettre l'environnement en état de fonctionnement :

```
sudo su - /etc/init.d/postgresql start
/etc/init.d/grafana-server start

# l'équivalent dans un environnement moderne
sudo systemctl start postgresql
sudo systemctl start grafana-server
```

Puis second étape pour notre première connexion à PostgreSQL :

```
sudo su - postgres psql
```

L'un des principes fondamentaux est qu'un utilisateur ne doit avoir que les droits nécessaires . Pour respecter les bonnes pratiques pour le bien de notre projet nous allons créer un rôle et lui donner les droits dont il a besoin . Notre utilisateur portera le nom de toto pour initialiser notre démarche .

```
# Création de l'utilisateur ( dans psql )
CREATE ROLE toto LOGIN PASSWORD 'mot_de_passe' ;

# Création de la base de donnée
psql -U toto -d postgres -f 01_create_db.sql

# Importation massif de donnée depuis un fichier
psql -U toto -d brigade_criminelle -f 02_import_data-1.sql
psql -U toto -d brigade_criminelle -f 03_import_massif.sql

# Connection a la base de donnée
psql -U toto -d brigade_criminelle
```

III – Requêtes SQL d'exploitation

Maintenant notre environnement bien installé il nous faut construire la structure des requêtes qui nous permettra d'exploiter au mieux nos données . Pour ce faire nous allons construire 6 requêtes qui mobilisent collectivement plusieurs principes du SQL primordiaux pour l'exploitation de données tels que les vues, sous-requêtes , jointure , agrégation ou encore les filtres .

```
– Nombre d'affaires par type et par statut :

SELECT af.type , af.statut , COUNT( af.dossier )
FROM affaire af
GROUP BY af.type , af.statut ;
```

Cette requête permet d'avoir une vision globale des affaires en comptabilisant celles-ci, qu'elles soient au statut « en cours », « fermée » ou « ouverte ».

Cette requête effectue un regroupement (GROUP BY) sur les colonnes « type » et « statut » de la table « affaire », puis compte (COUNT) le nombre de dossiers pour chaque combinaison de type et de statut.

- Agents ayant traité le plus grand nombre d'affaires :

```
SELECT agm.nom , agm.prenom , agm.num_individu ,  
COUNT( DISTINCT af.dossier ) AS nb_affaire  
FROM nom_agent implique agm  
INNER JOIN affaire af ON agm.dossier = af.dossier  
GROUP BY agm.num_individu, agm.nom, agm.prenom  
ORDER BY nb_affaire DESC  
LIMIT 7 ;
```

Cette requête nous donne le top 7 des meilleurs agents, classés dans l'ordre du meilleur, relié à son nombre d'affaires résolues.

Cette requête effectue un filtrage sur les agents (filtrés sur le rôle « enquêteur ») et les joint à la table « affaire ». Elle compte le nombre de dossiers uniques (COUNT(DISTINCT)) attribués à chaque agent, trie par ordre décroissant (ORDER BY DESC) et ne garde que les 7 premiers résultats (LIMIT 7).

- Individus impliqués dans plus d'une affaire :

```
SELECT i.nom , i.prenom , i.num_individu , COUNT(*)  
FROM individu i  
INNER JOIN implique im ON i.num_individu = im.num_individu  
GROUP BY i.nom , i.prenom , i.num_individu  
HAVING COUNT(*) > 1 ;
```

Cette requête permet d'acquérir les individus qui sont apparus dans le plus d'affaires, qu'ils soient suspects, témoins ou autres.

Elle effectue une jointure entre la table des individus et celle de leurs implications dans les dossiers. Nous regroupons les résultats (GROUP BY) de celle-ci et filtrons pour ne garder que les individus impliqués dans au moins un dossier d'affaire.

- Nombre moyen de pièces à conviction par affaire :

```
SELECT AVG ( nb_piece_moy ) FROM (
  SELECT af.dossier , COUNT( p.num_piece ) AS nb_piece_moy
  FROM affaire af
  LEFT JOIN piece p ON p.dossier = af.dossier
  GROUP BY af.dossier
);
```

Cette requête donne la moyenne des indices potentiels dans l'ensemble des affaires retenues par notre brigade criminelle.

Elle utilise une sous-requête. Elle se compose d'une requête interne qui calcule le nombre de pièces pour chaque dossier (l'utilisation du LEFT JOIN pour ne pas omettre les affaires sans pièce, NULL). La requête externe calcule ensuite la moyenne globale (AVG) sur ces résultats.

- Affaires ouvertes depuis plus de 6 mois sans clôture :

```
SELECT
  dossier ,
  date_debut ,
  type ,
  statut
FROM affaire
WHERE date_fin IS NULL AND statut <> 'close'
AND date_debut < NOW() - INTERVAL '6 months' ;
```

Cette requête effectue l'arborescence des dossiers d'affaires de plus de 6 mois, donnant au commandement les dossiers qui devraient être priorités.

Cette requête filtre les dossiers de la table « affaire » selon trois critères cumulatifs (AND) : l'absence de date de fin, un statut non ferme (donc non résolu) et une date de début antérieure de 6 mois à la date actuelle (NOW() - INTERVAL '6 months').

- Répartition géographique des affaires par département :

```
SELECT af.departement_code , d.nom_departement , d.latitude , d.longitude, COUNT(*) AS
total_affaires
FROM affaire af
JOIN departement_geo d ON d.code_departement = af.departement_code
GROUP BY af.departement_code, d.nom_departement, d.latitude , d.longitude ;
```

Cette requête donne le taux de criminalité de chaque département, servant à visionner dans quel département nous pourrions allouer plus d'effectifs.

Elle joint la table des affaires à une table de département géographique (departement_geo) afin de récupérer les coordonnées (latitude/longitude) et le nom du département, puis compte les affaires par zone géographique.

IV – Création de vues SQL

Une vue SQL représente une table virtuelle définie par une requête prédéfinie . Ce concept permet de faciliter et structurer l'accès aux données. Les vues proviennent du résultat d'une sélection effectuée sur une ou plusieurs tables, simplifiant la gestion de données complexes ou la sécurisation d'informations sensibles .Ses principaux avantages sont :

- Simplification des requêtes : Les vues permettent de masquer la complexité des requêtes sous-jacentes, facilitant ainsi la sélection des données à afficher pour l'utilisateur final.
- Sauvegarde de la définition : La description de la vue est stockée dans la base de données, ce qui facilite sa gestion.

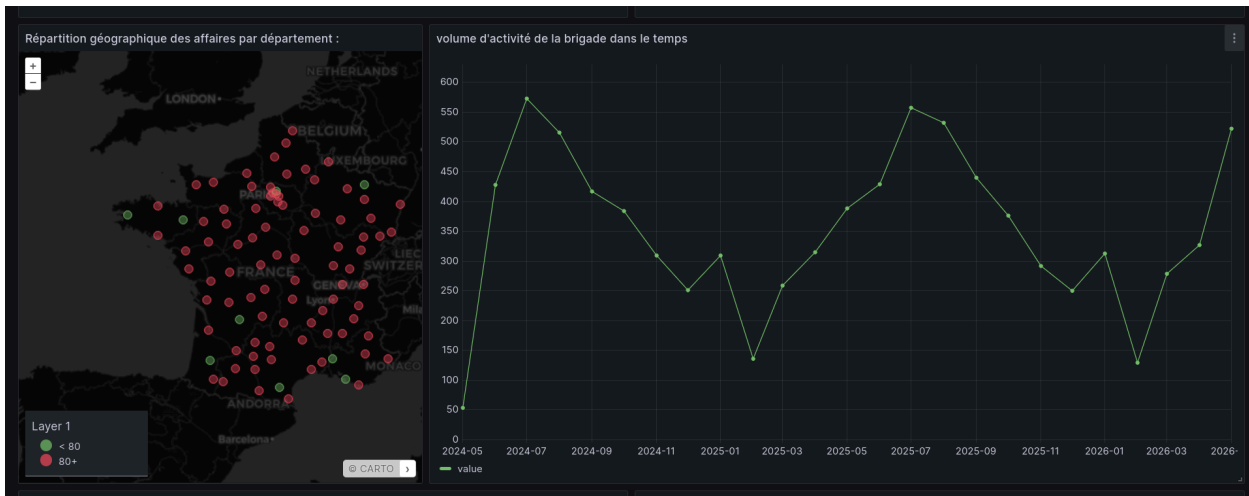
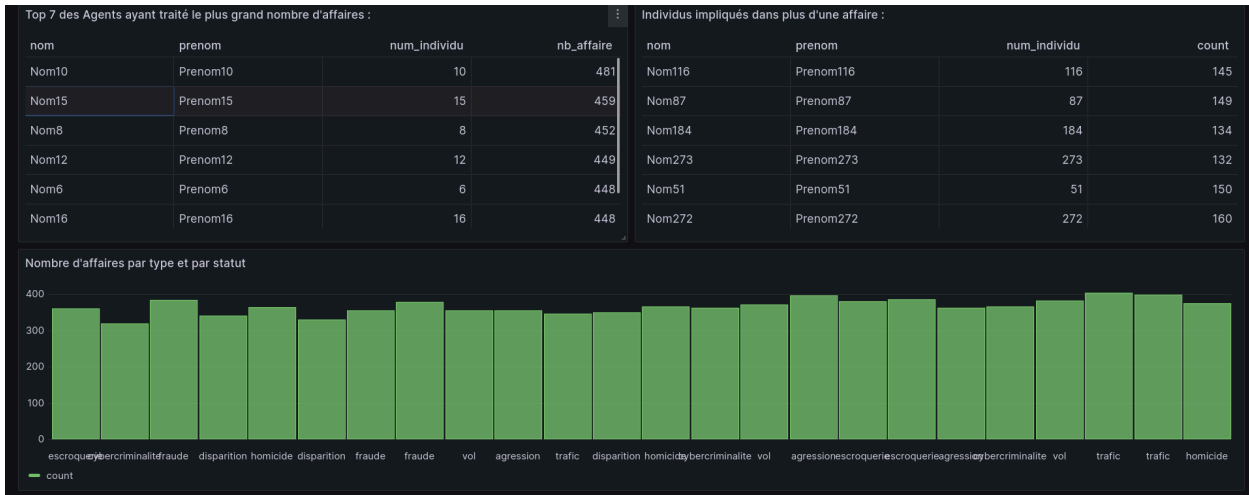
```
CREATE VIEW nom_agent_implique AS (  
  SELECT i.nom , i.prenom , i.num_individu , im.dossier  
  FROM individu i  
  INNER JOIN agent a ON i.num_individu = a.num_individu  
  INNER JOIN implique im ON im.num_individu = a.num_individu  
  WHERE im.role='enqueteur'  
);
```

Si vous avez pu y prêter attention, cette vue a déjà été utilisée auparavant pour construire la seconde requête dans la section : III – Requetes SQL d'exploitation.

Cette vue nous permet d'allier plusieurs tables entre elles pour faciliter l'accès à l'information, tel qu'ici le nom et le prénom d'un agent qui est enquêteur sur un dossier d'affaire en lien avec son numéro d'individu.

V – Construction du dashboard Grafana

Nous arrivons maintenant à la visualisation des données exploitées avec les requêtes préalablement créées à l'aide de Grafana. Cet outil open source facilite la création de tableaux de bord pour surveiller et interpréter les données.



VI – Sauvegarde PostgreSQL

Pour garantir la l'archivage des données et figer l'état final de la base de données brigade_criminelle, la commande de sauvegarde logique suivante a été exécutée dans le terminal :

```
# sauvegarde logique de la base brigade_criminelle dans son état final
pg_dump -U toto -d brigade_criminelle > sauvegarde_brigade_finale.sql

# lister le fichier avec plus de détaille
ls -lh sauvegarde_brigade_finale.sql
```

Cette sauvegarde logique est plutôt utile puisqu'elle est portable, car elle repose sur du texte brut au format SQL standard. Cependant, cela peut poser problème d'un point de vue sécurité. En effet, puisqu'elle repose sur du texte brut, elle n'est donc pas protégée par différents chiffrement. Celle-ci peut être lue et comprise par toute personne, d'autant plus qu'elle retranscrit des données sensibles, telles que l'architecture exacte de notre base de données, mais aussi les données intégrées, telles que, dans notre cas, le nom, le prénom d'individus ou encore des détails sur des dossiers d'affaires qui ne doivent être rendus publics. Ce fichier doit absolument être conservé dans de bonnes conditions .

Enfin la restauration de la base de données se fait par l'intermédiaire du fichiers texte créés par pg_dump qui peuvent être lus par le programme psql. La syntaxe générale d'une commande de restauration est :

```
# Par ailleurs il faut au préalable supprimer et recréer la base de donnée brigade_criminelle
psql brigade_criminelle < sauvegarde_brigade_finale.sql
```

VII – Conclusion

Ce projet, parcourant le monde des bases de données appliqué à une brigade criminelle, m'a permis de construire une chaîne de traitement de données complète. Notre architecture de données se repose sur une structure de traitement qu'est :

Application ⇒ PostgreSQL ⇒ Requêtes SQL & Vues ⇒ Grafana ⇒ Tableaux de bord

Tout au long de la réalisation de ce projet, j'ai pu concrétiser mes compétences et pu les consolider notamment grâce à l'utilisation d'outils tels que Grafana pour la visualisation des données ou encore aux requêtes SQL avancer construite pour celle-ci. Cette réalisation a pu être menée à bien par l'intermédiaire de Postgresql par laquelle j'ai pu stocker les données intégrées dans ma base de données qu'est notre brigade_criminelle. Enfin, l'architecture de données a pu être construite en respectant les contraintes liées aux données sensibles, mais aussi appliquer aux différentes contraintes métiers.

Enfin, notre dashboard amène d'excellentes bases opérationnelles. Ce projet m'a ouvert de nouvelles portes quant à l'amélioration de celle-ci notamment en intégrant de nouvelles technologies et en renforçant celles déjà utilisées. C'est pourquoi dans un futur proche, je construirai un projet de plus haute ampleur utilisant les mêmes types d'outils afin de pouvoir analyser les flux de carburant des stations-service en France et pouvoir comparer celle-ci afin de mettre en avant par différents algorithmes les meilleures stations de France.